

Informe de Explotación de Buffer Overflow: Un Estudio Didáctico de la Vulnerabilidad Minishare

Introducción

Estimado estudiante,

Como parte de tu formación en ciberseguridad, la comprensión de vulnerabilidades fundamentales como el *buffer overflow* es esencial. Esta técnica no solo representa una de las fallas de seguridad más antiguas y persistentes, sino que también sirve como una ventana al funcionamiento interno de las aplicaciones, la arquitectura de la memoria y la lógica del sistema operativo. Un *buffer overflow* no es simplemente un error que causa que un programa falle; es un vector de ataque que, si se manipula correctamente, puede otorgar a un atacante un control total sobre un sistema.

Este informe ha sido diseñado para servirte como una guía exhaustiva, combinando una sólida base teórica con la documentación de un caso práctico real. A lo largo de sus secciones, exploraremos los conceptos que rigen la pila de memoria, desglosaremos el proceso metódico para explotar una vulnerabilidad en la aplicación *Minishare* y, finalmente, analizaremos las defensas modernas que han sido desarrolladas para mitigar esta clase de ataques. El objetivo no es solo que aprendas a replicar una técnica, sino que desarrolles el pensamiento crítico necesario para identificar, analizar y defenderte de amenazas basadas en la manipulación de la memoria.

1. Fundamentos Teóricos del Buffer Overflow

1.1. ¿Qué es un Buffer Overflow? Una Anomalía de la Memoria

Un *buffer overflow*, también conocido como *buffer overrun*, es una anomalía en el software que ocurre cuando un programa intenta escribir datos en un espacio de memoria temporal (*buffer*) que es más pequeño que la cantidad de datos que se le suministra. Como

consecuencia, el exceso de información "desborda" el límite del *buffer* y sobrescribe las ubicaciones de memoria adyacentes.¹ Este error de programación es un subproducto de una mala práctica fundamental: la falta de "chequeo de límites" (*bounds checking*). En lenguajes de bajo nivel como C, que priorizan el rendimiento y el control sobre la seguridad, es responsabilidad del programador garantizar que las entradas de datos no excedan la capacidad de los *buffers*, una tarea que a menudo se pasa por alto.³

Las consecuencias de este desbordamiento varían. En el mejor de los casos, puede provocar un fallo del sistema, llevando a una denegación de servicio (DoS) y a la corrupción de datos, lo que se manifiesta en el sistema operativo con un mensaje de "la aplicación ha dejado de funcionar".¹ Sin embargo, en un escenario malicioso, un atacante puede manipular este desbordamiento para sobrescribir áreas críticas de la memoria del programa, lo que le permite ejecutar código arbitrario y, en última instancia, tomar el control del proceso.¹

Existen dos tipos principales de *buffer overflows*, clasificados según la ubicación de la memoria afectada:

- **Basados en la Pila (*Stack-based*):** Es el tipo más común y, generalmente, el más sencillo de explotar. Ocurre en la pila de llamadas del programa, donde se almacenan variables locales y la dirección de retorno de las funciones.¹ Al sobrescribir esta dirección, un atacante puede redirigir el flujo de ejecución del programa.
- **Basados en el Montón (*Heap-based*):** Son más complejos de ejecutar. Se producen en el área de memoria utilizada para la asignación dinámica de datos (*heap*). La explotación de esta vulnerabilidad se centra en corromper estructuras de datos internas del programa, como los punteros de listas enlazadas, para lograr la ejecución de código.¹

1.2. La Arquitectura de la Pila (Stack) de Memoria: El Campo de Batalla

Para comprender un ataque de *stack-based buffer overflow*, es imperativo entender la arquitectura de la pila de memoria. La pila es una región de memoria que opera bajo una estructura de datos LIFO (*Last In, First Out*), lo que significa que el último elemento añadido es el primero en ser retirado.⁶ Las aplicaciones utilizan esta memoria para gestionar las llamadas a funciones. Cada vez que se llama a una función, se crea una nueva estructura en la pila, conocida como *stack frame*.

La anatomía de un *stack frame* es crucial en este tipo de ataques, ya que el desbordamiento sigue una ruta predecible que recorre sus componentes.⁴ La pila crece desde direcciones de memoria altas hacia direcciones bajas, organizando sus elementos de la siguiente manera:

- **Variables Locales:** Se ubican en la parte más alta del *stack frame* y se utilizan para almacenar datos específicos de la función. En el caso de un *buffer overflow*, una entrada

maliciosa comienza a sobrescribir esta área.⁴

- **Puntero de Marco Guardado (*Saved Frame Pointer* o **EBP**):** Este puntero almacena la dirección del inicio del *stack frame* de la función que realizó la llamada. Sobrescribirlo es el siguiente paso en la cadena de un *overflow*.⁴
- **Dirección de Retorno (*Return Address* o **EIP**):** Es el elemento más crítico de la pila. Contiene la dirección de memoria a la que el procesador debe "saltar" una vez que la función actual finaliza. El objetivo de un atacante es sobrescribir esta dirección para redirigir el flujo de ejecución a un código malicioso de su elección.⁴

El desbordamiento de un *buffer* en la pila se convierte en un ataque explotable cuando los datos inyectados, al exceder el espacio de las variables locales, sobrescriben progresivamente el EBP y, finalmente, el EIP. Al tomar el control de la dirección de retorno, el atacante puede forzar al procesador a ejecutar un código malicioso, en lugar de retornar a la dirección esperada.⁴

1.3. El Mecanismo del Desbordamiento en la Pila: Del Crash al Control

El proceso para convertir una falla de programación en un vector de ataque es una secuencia de eventos meticulosamente planeada. El primer paso, y el más fundamental, es provocar un fallo intencional. Esto se logra enviando una cadena de caracteres tan larga que el programa no puede procesarla correctamente. La inyección de una cadena de texto excesivamente larga, por ejemplo, compuesta únicamente por el carácter 'A', inicia un desbordamiento. Esta cadena, al no caber en el *buffer* asignado, comienza a sobrescribir las variables adyacentes hasta que alcanza la dirección de retorno (EIP). Cuando la función termina, el procesador intenta saltar a la dirección que se encuentra en el EIP, que en este caso está llena de los bytes del carácter 'A' (hexadecimal 41414141). Dado que 41414141 no es una dirección de memoria válida, el sistema operativo no sabe qué hacer y detiene el programa, lo que resulta en un *crash*.⁴

Este *crash* es el primer "éxito" de un atacante, ya que confirma que el *buffer* es vulnerable y que es posible sobrescribir la memoria del programa, incluyendo su EIP.⁴ A partir de este punto, el atacante puede comenzar a perfeccionar el *exploit*, pasando del caos de un fallo incontrolado a la precisión necesaria para la ejecución de código. El siguiente paso en esta cadena lógica es determinar el número exacto de bytes necesarios para llegar al EIP sin sobrepasarlo.

2. Caso Práctico - Explotación de la Vulnerabilidad Minishare

A continuación, se presenta un caso práctico basado en la explotación de la aplicación vulnerable *Minishare* en un entorno controlado, siguiendo las etapas descritas en el material de investigación.⁴ El laboratorio consiste en una máquina virtual Kali Linux como atacante y una máquina virtual Windows XP como víctima, con el servidor *Minishare* corriendo en el puerto 80.

2.1. Fase 1: Provocando el Crash - Fuzzing para la Negación de Servicio (DoS)

2.1.1. Preparación del Laboratorio de Ataque

El primer paso para el desarrollo de un *exploit* es abandonar el entorno cliente-servidor tradicional del navegador. Un navegador web no ofrece el control granular necesario para manipular las peticiones a nivel de bytes, que es lo que se requiere en un ataque de *buffer overflow*. Para ganar este control, se utiliza una herramienta de proxy web como Burp Suite para interceptar y analizar las peticiones HTTP que el navegador envía al servidor.⁴ Una vez que se entiende la estructura de la petición (por ejemplo, GET /readme.txt HTTP/1.1), se construye un *framework* de *exploit* programable utilizando un lenguaje como Python y la librería socket.⁴ Este script inicial actúa como un "caparazón" que puede replicar las peticiones del navegador y modificarlas a voluntad.⁴

2.1.2. La Estrategia del Fuzzing

Con el script en Python listo, el siguiente paso es identificar el "punto de inyección", que es el lugar en la petición donde la aplicación espera una cadena de texto (un *string*), como un nombre de archivo.⁴ En el caso de *Minishare*, este punto corresponde al nombre del recurso en la petición GET.

El ataque comienza con una técnica llamada *fuzzing*, que consiste en enviar datos aleatorios o inesperados a una aplicación para observar su reacción.⁴ En este contexto, el *fuzzing* es un proceso metódico. Se utiliza el script de Python para enviar un número creciente de caracteres de un tipo específico, en este caso, el carácter 'A' (\x41 en ASCII). Se prueba con una cantidad pequeña de caracteres y, si la aplicación no se cae, se incrementa la cantidad (p.ej., 100, 500, 1000) hasta que se produce un fallo.⁴ La elección del carácter 'A' no es casual; su valor hexadecimal es fácil de identificar en un depurador, lo que simplifica el análisis posterior.

En el caso práctico, el proceso de *fuzzing* reveló que la aplicación *Minishare* era sorprendentemente robusta, ya que no colapsó con 1000 caracteres, pero sí lo hizo al alcanzar los 2000.⁴ Un análisis más detallado del rango (1500-2000) mostró que el *crash* se producía entre 1700 y 1800 caracteres.⁴ Este primer fallo, que provoca una denegación de servicio (DoS), es crucial porque confirma la vulnerabilidad y establece un rango de búsqueda para la siguiente fase.

2.1.3. Estructura del Payload - Fuzzing

La inyección de un patrón predecible, como una cadena de 'A's, es el primer paso en un proceso de validación del control. El fallo del programa no es el objetivo final, sino que funciona como una herramienta de diagnóstico. El hecho de que el EIP se sobrescriba con 41414141 (el valor de cuatro 'A's) en el informe de error de Windows confirma que el atacante ha logrado controlar la dirección de retorno, lo que abre la puerta a una explotación más avanzada.⁴

2.2. Fase 2: Determinando la Posición Exacta del EIP (Offset)

El *fuzzing* solo nos proporciona un rango aproximado del tamaño del *buffer* (1700-1800 caracteres), pero para un *exploit* estable se necesita el número exacto. Aquí es donde entran en juego las herramientas de Metasploit.

2.2.1. Uso de Patrones de Metasploit

Para determinar el desplazamiento (*offset*) exacto, se utilizan dos herramientas: `msf-pattern_create` y `msf-pattern_offset`.

1. **Generación del patrón:** Se usa `msf-pattern_create -l 1800` para generar una cadena de 1800 caracteres que es única y no se repite. Este patrón se introduce en el script de Python en lugar de la cadena de 'A's.⁴
2. **Análisis del Crash:** El programa se ejecuta con el nuevo *payload*. Al colapsar, el informe de error de Windows mostrará un valor de desplazamiento que corresponde a una porción del patrón, por ejemplo, 36684335. Este valor no es aleatorio, sino que es el "código" que nos dice qué parte de nuestra cadena única sobrescribió el EIP en el momento del fallo.⁴
3. **Cálculo del Offset:** El valor (36684335) se utiliza con la herramienta `msf-pattern_offset -l 1800 -q 36684335` para calcular la posición exacta.⁴ La herramienta devuelve el valor de

1787, lo que indica que se necesitan exactamente 1787 bytes para llenar el *buffer* y llegar al inicio del EIP.⁴

2.2.2. Verificación del Control del EIP

Para verificar que el control es preciso, se modifica el script de Python. Ahora, el *payload* se construye con 1787 caracteres 'A', seguidos de 4 caracteres 'B' (\x42), ya que las direcciones de 32 bits ocupan 4 bytes, y el resto con caracteres 'C' (\x43). Cuando se ejecuta este *payload*, la aplicación vuelve a colapsar, pero el informe de error muestra 42424242 en el EIP (el valor hexadecimal de cuatro 'B's').⁴ Este resultado confirma el control preciso sobre el EIP.

La tabla a continuación muestra la evolución de la construcción del *payload* durante estas primeras fases.

Fase del Exploit	Estructura del Payload	Valor del EIP en el Crash	Propósito
Paso 1: Fuzzing	GET + A * 2000 + HTTP/1.1\r\n\r\n	41414141	Causa un DoS para confirmar la vulnerabilidad y el control del EIP.
Paso 2: Offset	GET + Patrón * 1800 + HTTP/1.1\r\n\r\n	36684335	Encuentra el desplazamiento (1787) exacto del EIP con un patrón único.
Paso 3: Verificación	GET + A * 1787 + B * 4 + C * 400 + HTTP/1.1\r\n\r\n	42424242	Confirma que se puede sobrescribir con precisión el EIP con un valor conocido.

2.3. Fase 3: Identificación de Caracteres Defectuosos (Bad Chars)

Una vez que se tiene un control preciso sobre el EIP, el siguiente obstáculo es asegurarse de que el código que se va a inyectar (*shellcode*) no contenga "caracteres defectuosos" (*bad chars*). Estos son bytes que el programa vulnerable o el sistema operativo interpretan de forma especial, interrumpiendo o corrompiendo nuestra *shellcode*.⁴ Un ejemplo común es el byte nulo (\x00), que en lenguajes como C es un terminador de cadena.⁷ Si se introduce este carácter, el programa detendrá el procesamiento del *payload* en ese punto, haciendo que la *shellcode* sea inútil.

2.3.1. Metodología de Detección

Para identificar estos caracteres, se sigue una metodología de prueba y error combinada con un depurador.

1. **Generar la cadena de prueba:** Se crea un script de Python que genera una cadena que contiene todos los bytes posibles de \x01 a \xff.⁴
2. **Inyectar la cadena de prueba:** Se inyecta esta cadena en la parte del *payload* que sigue a la sobrescritura del EIP (la sección de las 'C's').⁴
3. **Depurar la aplicación:** El programa vulnerable (*Minishare*) se ejecuta dentro de un depurador (como Immunity Debugger). Cuando el programa colapsa, el depurador muestra el estado de la memoria en ese momento. Se inspecciona la pila (utilizando el registro ESP) para ver los bytes que la aplicación ha recibido.⁴
4. **Comparar y refinar:** Se compara la secuencia de bytes inyectada con la que se observa en la memoria. Si un byte de la secuencia original no aparece, o si los bytes que lo siguen están desorganizados, se ha encontrado un *bad char*. El proceso es iterativo: se elimina el carácter defectuoso de la cadena de prueba y se vuelve a ejecutar hasta que la secuencia de bytes en la memoria coincide exactamente con la que se inyectó.⁴

2.3.2. Caracteres Defectuosos Identificados

El proceso de depuración en *Minishare* reveló que los siguientes caracteres son defectuosos y deben ser excluidos de cualquier *shellcode*:

Carácter	Valor Hexadecimal	Razón del Problema
Nulo	\x00	Actúa como un terminador de cadena en C/C++, deteniendo el

		procesamiento del <i>payload</i> .
Retorno de carro	\x0D	Un carácter especial que es filtrado o interpretado de forma anómala por el programa, corrompiendo la secuencia de bytes.

El proceso de identificación de los *bad chars* demuestra que la explotación de una vulnerabilidad es un proceso de validación riguroso por parte del atacante, que debe trabajar dentro de las reglas del sistema. El depurador actúa como una herramienta de "rayos X" que permite al atacante ver cómo el sistema interpreta su entrada maliciosa, lo que es esencial para la estabilidad del *exploit* final.

2.4. Fase 4: Obtención de Acceso Remoto

La fase final une todos los hilos del proceso para lograr el objetivo final: obtener una *reverse shell* (acceso remoto) al sistema.

2.4.1. El Salto a la Shellcode (JMP ESP)

El *exploit* final debe hacer más que sobrescribir el EIP con un valor que cause un fallo. Necesita una dirección que, al ser ejecutada, redirija el flujo a nuestra *shellcode*. La instrucción ideal para este propósito es JMP ESP (*Jump to ESP*). Esta instrucción hace que el procesador salte a la dirección de memoria apuntada por el registro ESP, que en este punto de la ejecución del *exploit*, apunta al inicio de nuestra *shellcode* en la pila.⁴

Para encontrar una instrucción JMP ESP, se utiliza el depurador para buscar en las bibliotecas de enlace dinámico (*DLLs*) del sistema operativo. Se buscan *DLLs* que sean "universales" (como user32.dll o kernel32.dll) porque suelen cargarse en la misma dirección de memoria en la mayoría de los sistemas, lo que hace que el *exploit* sea más estable y portable.⁴ La dirección de memoria de la instrucción

JMP ESP encontrada (0x7cb41020) se inserta en el *payload* en lugar de las 'B's de la fase de verificación. Es vital recordar el formato *little-endian* de las arquitecturas Intel, lo que requiere

que la dirección se escriba en orden inverso en el *payload*: \x20\x10\xb4\x7c.⁴

2.4.2. Generación de la Shellcode

La *shellcode* es el código malicioso que se inyectará. Para generarla, se utiliza la poderosa herramienta msfvenom de Metasploit. El comando se configura para crear un *payload* de tipo *reverse shell* de *meterpreter* que se conecte a la máquina del atacante (Kali Linux) en una dirección y puerto específicos.⁴ Es fundamental utilizar el parámetro -b para excluir los *bad chars* identificados previamente (\x00 y \x0d), asegurando que la *shellcode* no se corrompa al ser inyectada.⁴

La siguiente tabla desglosa el comando utilizado para generar la *shellcode*:

Parámetro	Valor Utilizado	Función / Explicación
-p	windows/meterpreter/reverse_tcp	Especifica el tipo de <i>payload</i> : un <i>reverse shell</i> para Windows que utiliza el protocolo TCP.
LHOST	192.168.92.130	La dirección IP del atacante, donde la <i>shell</i> de la víctima se conectará.
LPORT	4444	El puerto de escucha del atacante, donde se espera la conexión.
-b	"\x00\x0d"	La lista de "caracteres defectuosos" que msfvenom debe evitar al generar la <i>shellcode</i> .
-f	python	El formato de salida de la <i>shellcode</i> , listo para ser copiado en el script de Python.

2.4.3. Ensamblaje Final del Exploit y Lanzamiento

El *payload* final es una cuidadosa concatenación de todos los elementos:

1. **Buffer de Relleno:** Los 1787 caracteres `\x41` que llenan el *buffer*.
2. **Dirección JMP ESP:** La dirección de la instrucción de salto (`\x20\x10\xb4\x7c`) que sobrescribe el EIP.
3. **NOP Sled:** Una cadena de instrucciones NOP (*No Operation*, `\x90`) que actúan como un "colchón".⁴ Estas instrucciones no hacen nada, pero garantizan que si el salto no es perfectamente preciso, el flujo de ejecución "se deslice" por la cadena de NOPs hasta el inicio de la *shellcode*.²
4. **Shellcode:** La *shellcode* binaria generada por *msfvenom*.

Finalmente, en la máquina atacante se utiliza el módulo multi/handler de Metasploit para ponerse a la escucha en la dirección IP y puerto especificados.⁴ Al ejecutar el script de Python, el *payload* se envía a la aplicación *Minishare*, lo que provoca el desbordamiento. A diferencia del *fuzzing*, el programa no se detiene, sino que redirige su flujo de ejecución al JMP ESP, que a su vez salta al NOP sled y luego a la *shellcode*. Esto resulta en la apertura de una sesión de *meterpreter* en la máquina del atacante, otorgándole control remoto total sobre el sistema Windows XP.⁴

3. Hacia la Seguridad - Mitigaciones y Defensa

3.1. Defensas Modernas contra el Buffer Overflow

El *exploit* detallado en este informe es una técnica clásica que, si bien funciona en sistemas antiguos como Windows XP, rara vez tiene éxito en entornos operativos modernos. Los desarrolladores y los proveedores de sistemas operativos han implementado una serie de mitigaciones que hacen que la explotación sea significativamente más difícil.³

- **Prevención de Ejecución de Datos (*Data Execution Prevention* o DEP):** Esta característica del sistema operativo marca la pila de memoria como "no ejecutable".⁷ En sistemas con DEP, el procesador no puede ejecutar código desde la pila, lo que frustra el

ataque de
stack-based buffer overflow.

- **Aleatorización del Espacio de Direcciones (*Address Space Layout Randomization* o **ASLR**):** ASLR aleatoriza las direcciones en las que se cargan los componentes de un programa (como las *DLLs* y el *stack*). Esto significa que la dirección de una instrucción *JMP ESP* cambia cada vez que se ejecuta el programa, invalidando la dirección estática que un atacante podría haber identificado previamente.⁸ Esto elimina la predictibilidad del ataque.
- **Canarios de Pila (*Stack Canaries*):** Los *stack canaries* son valores aleatorios que se colocan en la pila justo antes de la dirección de retorno de una función.⁸ Antes de que una función retorne, el programa verifica si el valor del canario ha cambiado. Si ha sido sobrescrito por un *buffer overflow*, el programa detecta el ataque y aborta la ejecución para prevenir la explotación, lo que frustra el intento de un atacante de alcanzar la dirección de retorno.⁷

3.2. Conclusión y Recomendaciones para un Futuro Analista de Ciberseguridad

El estudio de los ataques de *buffer overflow* es una pieza fundamental en la formación de un analista de ciberseguridad. Lo que has aprendido aquí va más allá de la simple ejecución de un *script*. La verdadera lección es la cadena de pensamiento que subyace al ataque: cómo un atacante utiliza un fallo del sistema para recopilar información, validar su hipótesis de ataque y construir un *payload* que manipule el flujo de ejecución del programa.

Este ejercicio, que utiliza un sistema operativo y una aplicación antiguos, demuestra por qué las mitigaciones modernas son tan importantes y necesarias. Un atacante no se detiene en la primera barrera; por el contrario, busca cómo sortearla. Al comprender cómo funciona un ataque clásico, un analista puede entender mejor las limitaciones de las defensas modernas (por ejemplo, cómo se pueden "filtrar" los valores de los *canaries*) y, lo que es más importante, puede identificar los errores de codificación que dan origen a estas vulnerabilidades en primer lugar.⁸

Te recomiendo que no te detengas en este punto. Explora otras clases de vulnerabilidades de memoria (como los *heap overflows* y los *format string attacks*), investiga las técnicas de evasión de las mitigaciones actuales y, lo más importante, profundiza en las prácticas de codificación segura para prevenir estos fallos desde el origen. La mejor defensa contra un ataque de *buffer overflow* es asegurarse de que la vulnerabilidad nunca exista.

Obras citadas

1. What Is Buffer Overflow? Attacks, Types & Vulnerabilities - Fortinet, fecha de

acceso: septiembre 6, 2025,

<https://www.fortinet.com/resources/cyberglossary/buffer-overflow>

2. Buffer overflow - Wikipedia, fecha de acceso: septiembre 6, 2025,
https://en.wikipedia.org/wiki/Buffer_overflow
3. Una solución estática al problema de Buffer Overflow - SEDICI, fecha de acceso: septiembre 6, 2025,
http://sedici.unlp.edu.ar/bitstream/handle/10915/21929/Documento_completo.pdf?sequence=1&isAllowed=y
4. AS-II - Buffer Overflow.pdf
5. Stack buffer overflow - Wikipedia, fecha de acceso: septiembre 6, 2025,
https://en.wikipedia.org/wiki/Stack_buffer_overflow
6. 3.2.9 DIRECCIONAMIENTO DE LA PILA DE MEMORIA - Wix.com, fecha de acceso: septiembre 6, 2025,
<https://franck03.wixsite.com/arquitecturafranceli/329-direccionamiento-de-la-pila-de-mem>
7. Buffer overflow protection - Wikipedia, fecha de acceso: septiembre 6, 2025,
https://en.wikipedia.org/wiki/Buffer_overflow_protection
8. What Are Stack Canaries? How are they Preventing Overflow? - IO River, fecha de acceso: septiembre 6, 2025, <https://www.ioriver.io/terms/stack-canaries>
9. Protecting Your Stack: The Importance of Stack Canaries in Security - Blue Goat Cyber, fecha de acceso: septiembre 6, 2025,
<https://bluegoatcyber.com/blog/protecting-your-stack-the-importance-of-stack-canaries-in-security/>